

IR

IMDb

Internet Movie Database

Group 5

李宇捷 (Ethan) 01057108

沈奎勳 (Joseph) 01057115

陳柏瑋 01057138

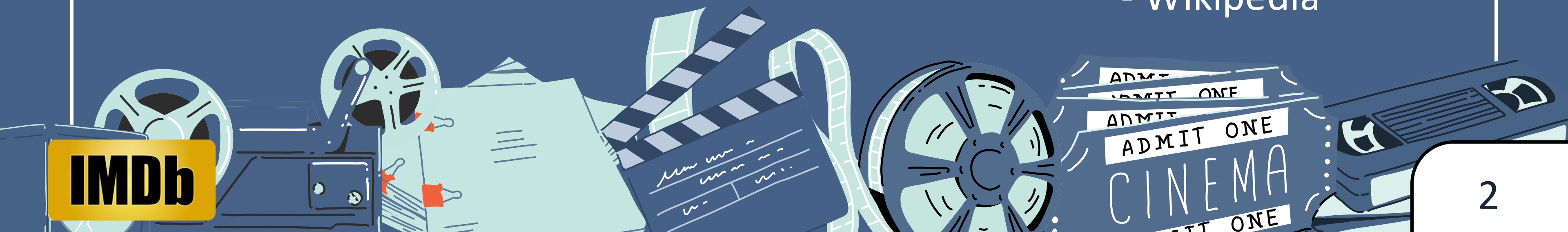
洪旻昌 (Bryan) 01057158

IMDb

What is IMDb?

The Internet Movie Database (IMDb) is an online database of information related to films, television series, podcasts, home videos, video games, and streaming content online – including cast, production crew and personal biographies, plot summaries, trivia, ratings, and fan and critical reviews.

- Wikipedia



Functionality

1. Actor-Related Keywords

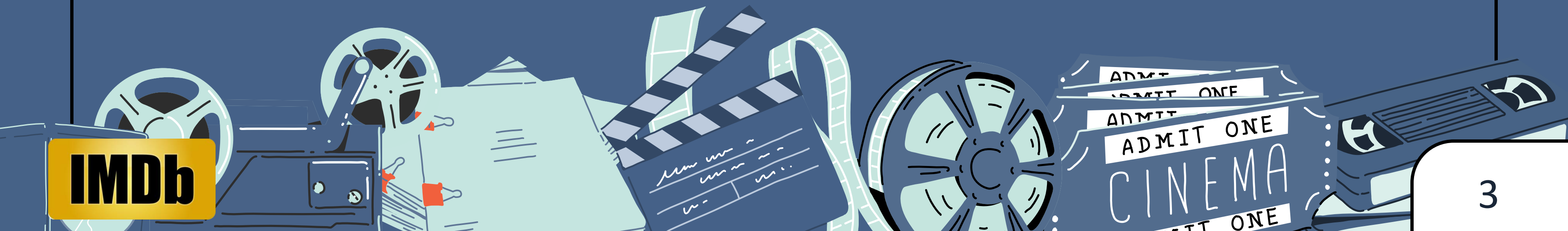
Display keywords based on reviews relevant to the actors.

2. Sentiment Grouping

Analyze and group user reviews based on each content's sentiments.

3. Review Summary

Construct a summarization of reviews for each rating level.



Movie List Retrieval

○ Retrieve movies' [ID] and [Title]



#movie list and ID
movies



```
[{'ID': 'tt0111161', 'name': 'The Shawshank Redemption'},  
{'ID': 'tt0068646', 'name': 'The Godfather'},  
{'ID': 'tt0468569', 'name': 'The Dark Knight'},  
{'ID': 'tt0071562', 'name': 'The Godfather Part II'},  
{'ID': 'tt0050083', 'name': '12 Angry Men'},  
{'ID': 'tt0167260', 'name': 'The Lord of the Rings: The Return of the King'},  
{'ID': 'tt0108052', 'name': 'Schindler's List'},  
{'ID': 'tt0110912', 'name': 'Pulp Fiction'},  
{'ID': 'tt0120737',  
  'name': 'The Lord of the Rings: The Fellowship of the Ring'},  
{'ID': 'tt0060196', 'name': 'Il buono, il brutto, il cattivo'},  
{'ID': 'tt0109830', 'name': 'Forrest Gump'},  
{'ID': 'tt0167261', 'name': 'The Lord of the Rings: The Two Towers'},  
{'ID': 'tt0137523', 'name': 'Fight Club'},
```

Review Retrieval

○ Retrieve movies' [Reviews]

```
1  {
2    "metadata": {
3      "movie_title": "Interstellar",
4      "movie_imdb_id": "tt0816692",
5      "reviews_count": 6724
6    },
7    "reviews": {
8      "1": {
9        {
10         "title": "vastly overrated",
11         "content": "reading the other reviews here its hard to conceive that I watched
12        },
13        {
14         "title": "Awful",
15         "content": "Everything about this movie is so bad, especially the acting.\n\nMa
16      },

```

Rating range: [1, 10]

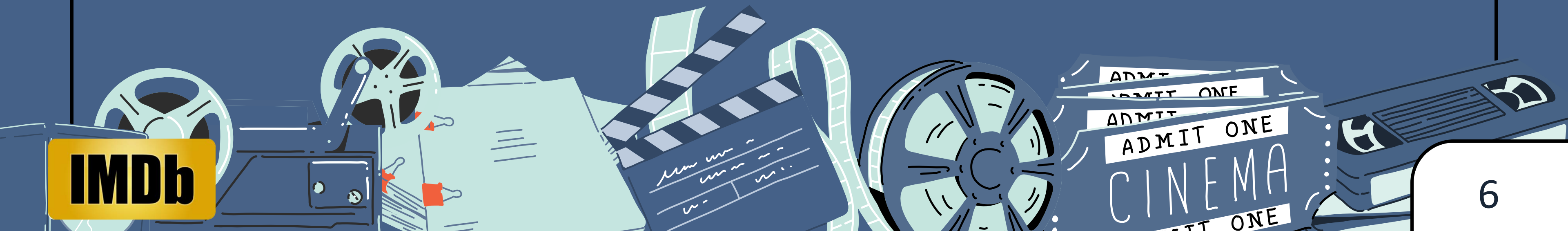
Collected Data

○ Total movie: 39 movies.

○ Total review: 92,621 reviews.

The least number of reviews: 436 reviews.

The most number of reviews: 10,837 reviews.



Database Construction

- Construct a local database to store reviews and its embedding into FAISS vector database. Model used: intfloat/multilingual-e5-large-instruct (embedding size = 1024)

```
def update_faiss_db(self):
    json_files = [f for f in os.listdir(self.upload_dir) if f.endswith(".json")]
    for json_file in json_files:
        with open(os.path.join(self.upload_dir, json_file), 'r', encoding='utf-8') as f:
            data = json.load(f)

        for score_str, reviews in data["reviews"].items():
            contents = [review["content"] for review in reviews]
            print(f"Score {score_str}: {len(contents)} reviews")
            vectorstore = FAISS.from_texts(contents, self.embedding_model)
            print(f"Saving to {os.path.join(self.db_dir, f'{json_file[:-5]}_{score_str}')}")
            vectorstore.save_local(os.path.join(self.db_dir, f'{json_file[:-5]}_{score_str}'))
```

#Functionality-1

Actor-Related Keywords



Actor-Related Keywords

Corpus Construction: Review Selection

```
{ TomCruise.json > ...  
{  
  "metadata": {  
    "movie_title": [  
      "Mission: Impossible – Rogue Nation",  
      "Top Gun: Maverick",  
      "Mission: Impossible – The Final Reckoning",  
      "Mission: Impossible",  
      "Mission: Impossible III",  
      "Edge of Tomorrow",  
      "Mission: Impossible – Fallout",  
      "Mission: Impossible – Dead Reckoning Part One",  
      "Oblivion",  
      "Top Gun",  
      "Mission: Impossible II",  
      "Mission: Impossible – Ghost Protocol"  
    ],  
    "movie_imdb_id": [...]  
  },  
  "complete_reviews": [...]  
  "split_reviews": [...]  
}
```

Query target words

- actor's name
- actor's role

complete_reviews: retrieve whole reviews that consist one of the query's target words.

split_reviews: retrieve sentences that consist one of the query's target words.

Actor-Related Keywords

Document Processing: Lemmatize, Stop Words, TF-IDF Vector

```
self.lemmatizer = WordNetLemmatizer()
self.stop_words = set(stopwords.words("english"))
self.stop_words.update(["movie", "film", "book", "mission", "impossible", "one", "like", "scene", \
                        "time", "really", "plot", "story", "see", "first", "part", "get", "much", \
                        "make", "also", "even", "mi", "watch", "still", "could", "way", "go", "think", \
                        "know", "lot", "would", "end", "seen", "new", "say", "made", "every", "never", \
                        "two", "woo", "thing", "feel", "10", "come", "act", "keep", "take", "read", \
                        "give", "fan", "leave", "many", "find", "little", "well", "seem", "year", \
                        "director", "show", "people", "though", "look", "last", "bite", "enjoy", \
                        "half", "want", "start", "back", "play", "actor", "actress", "cast", "knight"])

if self.folder_name.endswith("TomCruise.json"):
    self.stop_words.update(["tom cruise", "ethan hunt", "ethan", "hunt", "pete mitchell", "pete", "mitchell", \
                           "maverick", "cage", "jack", "tom", "cruise"])
elif self.folder_name.endswith("DanielRadcliffe.json"):
    self.stop_words.update(["daniel radcliffe", "harry potter", "harry", "potter", "daniel", "radcliffe"])
elif self.folder_name.endswith("MorganFreeman.json"):
    self.stop_words.update(["morgan freeman", "red", "redding", "lucius fox", "lucius", "fox", "morgan", "freeman", \
                           "beech", "thaddeus bradley", "thaddeus", "bradley", "president trumbull", \
                           "allan trumbull", "allan", "trumbull", "andy", "robbins", "shawshank", \
                           "bale", "tim", "caine", "batman", "joker", "michael", "bruce", "nolan", "wayne", "gary", "alfred"])

self.vectorizer = TfidfVectorizer(stop_words = list(self.stop_words), max_features=max_features, sublinear_tf=sublinear_tf)
```

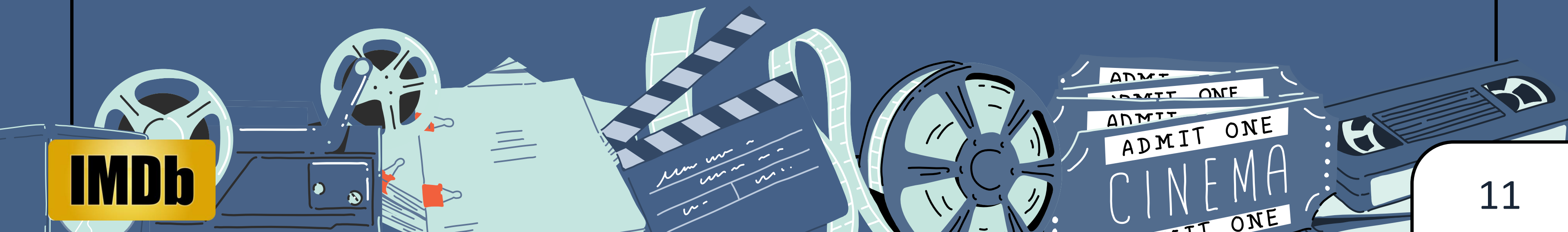
Actor-Related Keywords

Corpus Processing

```
with open(self.file_name, 'r', encoding='utf-8') as f:  
    data = json.load(f)  
  
self.reviews = data.get(self.kind_of_review, [])  
self.documents = [self.lemmatize_text(review) for review in self.reviews]
```

Retrieve Top-k Related Keywords

```
X = self.vectorizer.fit_transform(self.documents)  
scores = X.sum(axis = 0).A1  
terms = self.vectorizer.get_feature_names_out()  
sorted_idx = scores.argsort()[::-1]  
self.keywords = [(terms[i], scores[i]) for i in sorted_idx[:self.top_k]]
```



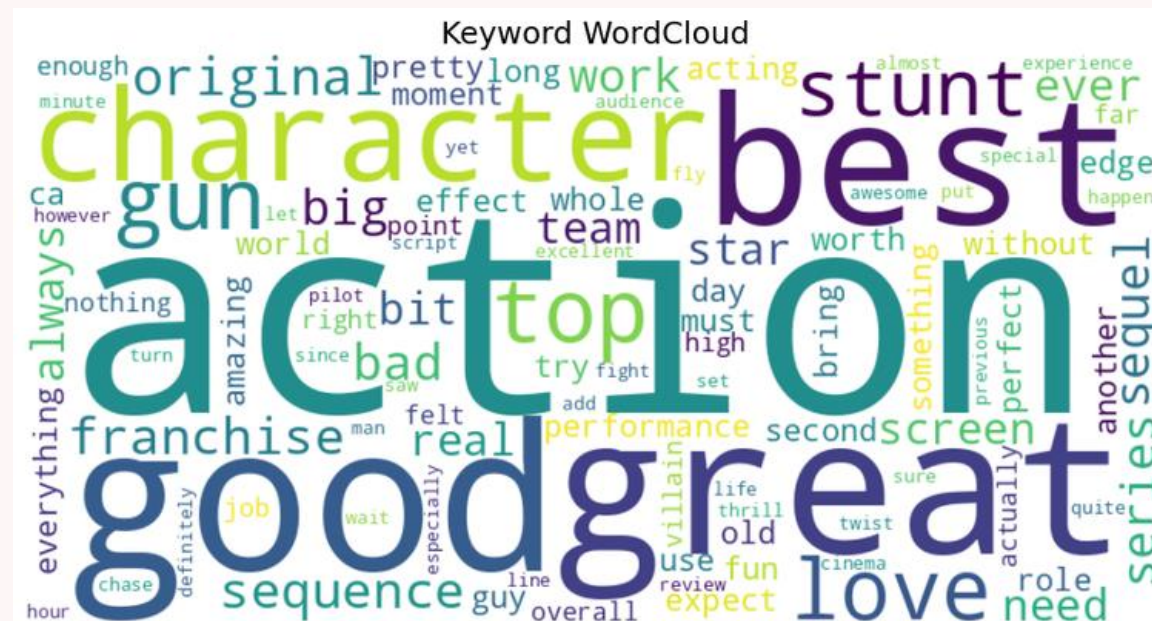
Actor-Related Keywords

Result of Top-20 Related Keywords

Actor: Tom Cruise

Source: 12 Movies with 15,922 Reviews

complete_reviews: 10,953 Data

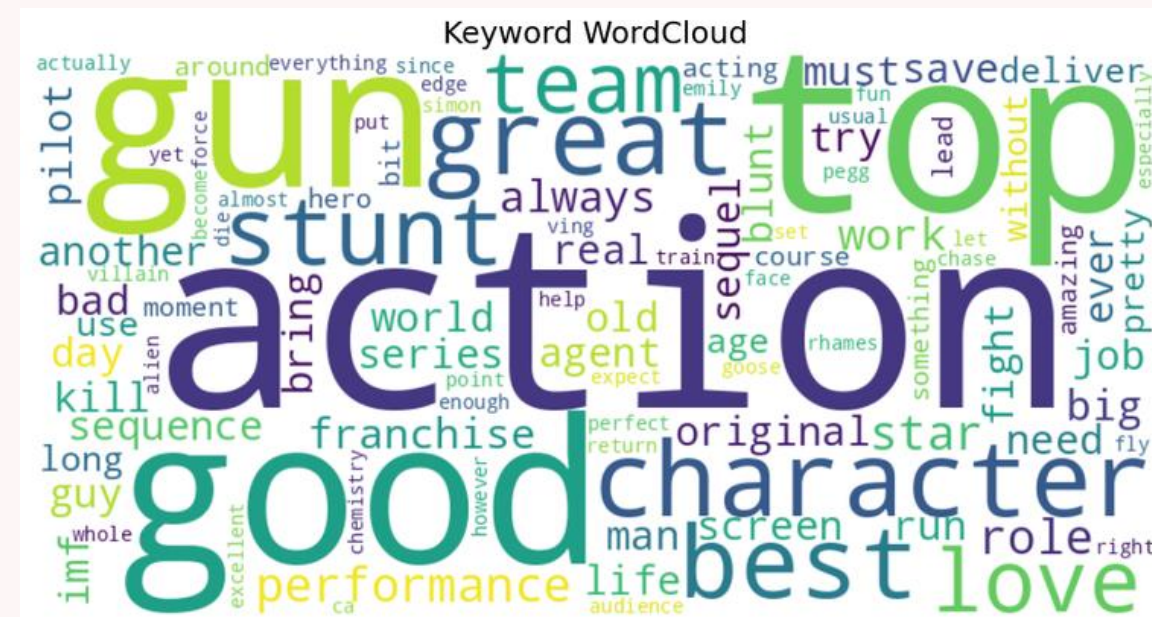


```

action: 403.21
good: 354.01
great: 309.68
best: 265.51
character: 259.22
top: 257.62
love: 229.77
gun: 229.44
stunt: 197.76
original: 183.95
sequence: 175.13
franchise: 173.81
bad: 166.50
series: 160.34
big: 159.24
sequel: 155.55
work: 147.55
ever: 138.76
need: 132.86
real: 131.24

```

split_reviews: 27,731 Data



```

action: 540.94
top: 505.15
good: 486.94
gun: 470.99
character: 429.42
great: 424.74
best: 389.25
stunt: 347.24
love: 299.52
team: 280.82
performance: 240.43
role: 239.28
star: 237.77
work: 211.17
always: 200.06
franchise: 192.37
big: 188.95
imf: 182.34
original: 181.01
sequence: 179.75

```

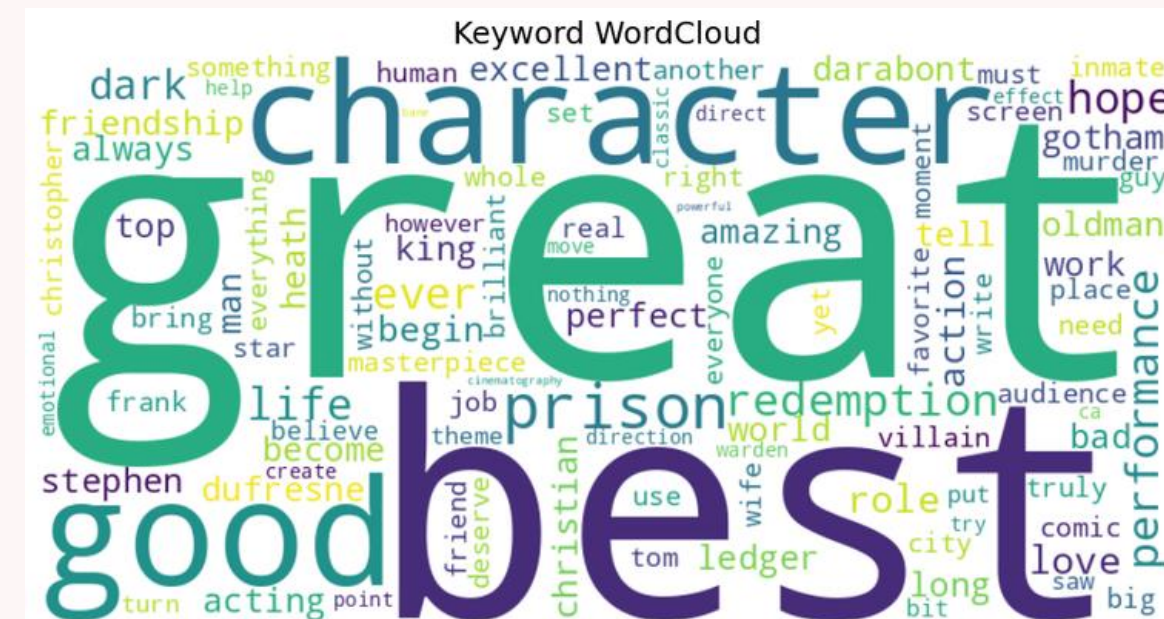

Actor-Related Keywords

Result of Top-20 Related Keywords

Actor: Morgan Freeman

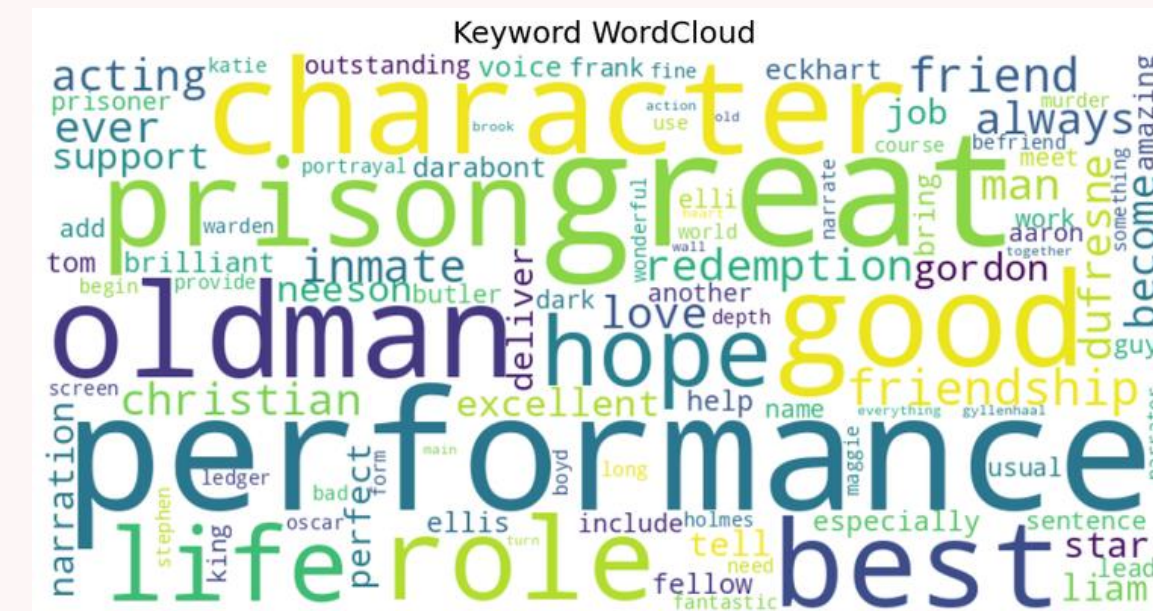
Source: 9 Movies with 21,233 Reviews

complete reviews: 7,653 Data



great: 242.43
best: 236.09
good: 232.20
character: 211.48
prison: 206.95
redemption: 189.38
performance: 185.91
life: 177.16
ever: 172.76
hope: 172.68
dark: 154.68
love: 136.78
role: 131.75
action: 124.41
begin: 117.66
man: 112.53
friendship: 109.04
christian: 107.57
ledger: 106.96
acting: 106.14

split reviews: 11,320 Data



```
performance: 288.79
great: 276.12
character: 232.24
good: 225.75
oldman: 225.29
prison: 225.23
best: 223.32
role: 202.81
life: 156.14
hope: 140.86
friendship: 136.90
friend: 125.28
inmate: 119.84
redemption: 112.61
man: 111.52
always: 110.51
christian: 107.44
ever: 104.62
love: 102.80
acting: 99.23
```


#Functionality-2

Group by Sentiment



Group by Sentiment

- Retrieve *tf.keras.datasets.imdb* data (as training and testing set) and manually create additional testing data.

Total: 25,000 data for training and 25,000 data for testing.

- Manually classify additional training data into two categories: “Positive” (1) and “Negative” (0).

Total: 300 data with 123 positive data and 177 negative data.

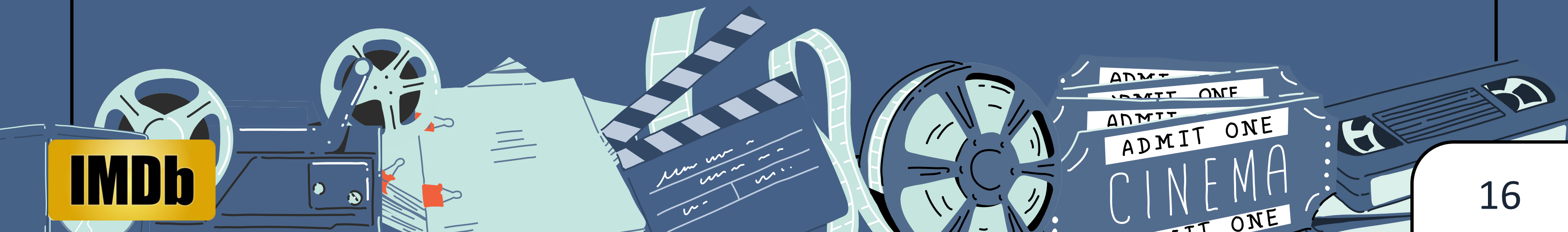
title	content	class
Cold and flat	The concept of Inception is a very appealing one, and the film is wonderfully inventive. It's also not as confusing or effects-heavy as some have said - the story is actually very well constructed. That just makes it all the more frustrating, because Inception has some very hard to define flaws that prevents it from being a truly great movie - most of all the very uninteresting characters. It's not exactly that they're flat - they're all convincing enough and the dialog is fluent enough, but none of them deserve any true emotion or interest from the viewer. It's a good movie by any measure, and yet - like every previous Christopher Nolan movie - stops just short of being a fantastic one.	0
Inceptionally	Watched by mistake and thoroughly glad I did, possibly predictable but my sort of thought provoking complexity that I like. The subliminal mind is a complex and profound construction. To be able to convey this via cinematography into a visual entertainment source really was an evenings worth of entertainment. The integral levels of dream scape and retaining the different levels of unconscious thought adds a fast paced eagerness to the film. It shoes elements o, is the end the beginning, is this the now? The film left me questioning and wanting to continue watching. I would I I love for their to be a second inception, this would have to be done in the correct manner as to keep the intrigue alive	1

Group by Sentiment

- Split training data into training and validation data to train the neural network.

```
x_tr, x_val, y_tr, y_val = model_selection.train_test_split(x_train, y_train,  
                                                           test_size=0.2, random_state=42,  
                                                           stratify=y_train)
```

- In the following two pages, the data from Keras is used as :
 - training,
 - validation, and
 - first testing



Group by Sentiment

- Trained 54 kinds of neural networks with different structures and/or hyperparameters.

```
structure = hp.Choice('structure', [
    '1x16', '1x32',
    '2x32_16', '2x32_16_d',
    '2x32_16_b', '2x32_16_db',
    '2x64_32', '2x64_32_d',
    '2x64_32_b', '2x64_32_db',
    '3x64_32_16', '3x64_32_16_d',
    '3x64_32_16_b', '3x64_32_16_db',
    '3x128_64_32', '3x128_64_32_d',
    '3x128_64_32_b', '3x128_64_32_db'
])
lr = hp.Choice('learning_rate', [0.0001, 0.001, 0.01])
```

```
if structure == '1x16':
    model.add(keras.layers.Dense(16, activation='relu'))
elif structure == '1x32':
    model.add(keras.layers.Dense(32, activation='relu'))
elif structure == '2x32_16':
    model.add(keras.layers.Dense(32, activation='relu'))
    model.add(keras.layers.Dense(16, activation='relu'))
elif structure == '2x32_16_d':
    model.add(keras.layers.Dense(32, activation='relu'))
    model.add(keras.layers.Dropout(0.2))
    model.add(keras.layers.Dense(16, activation='relu'))
elif structure == '2x32_16_b':
    model.add(keras.layers.Dense(32))
    model.add(keras.layers.BatchNormalization())
    model.add(keras.layers.Activation('relu'))
    model.add(keras.layers.Dense(16, activation='relu'))
elif structure == '2x32_16_db':
    model.add(keras.layers.Dense(32))
    model.add(keras.layers.BatchNormalization())
    model.add(keras.layers.Activation('relu'))
    model.add(keras.layers.Dropout(0.2))
    model.add(keras.layers.Dense(16, activation='relu'))
```

Group by Sentiment

- Performance of neural networks with best validation accuracy on both Keras and own test set.

```
Best val_acc model:
Structure: 2x64_32_db; Learning rate: 0.001
782/782 ————— 1s 1ms/step
10/10 ————— 0s 3ms/step
Test accuracy on keras test set: 0.95148
Test accuracy on self test set: 0.8833333333333333
Test confusion matrix on keras test set:
[[11831  669]
 [ 544 11956]]
Test confusion matrix on self test set:
[[147  30]
 [  5 118]]
```

Test classification report on keras test set:

	precision	recall	f1-score	support
0	0.96	0.95	0.95	12500
1	0.95	0.96	0.95	12500
accuracy			0.95	25000
macro avg	0.95	0.95	0.95	25000
weighted avg	0.95	0.95	0.95	25000

Test classification report on self test set:

	precision	recall	f1-score	support
0	0.97	0.83	0.89	177
1	0.80	0.96	0.87	123
accuracy			0.88	300
macro avg	0.88	0.89	0.88	300
weighted avg	0.90	0.88	0.88	300



#Functionality-3

Review Summary



Review Summary

- Use k-means clustering to split embedding into k-groups.

```
def kmeans_clustering(self, vectors):  
    kmeans = KMeans(n_clusters=self.k, random_state=42)  
    kmeans.fit(vectors)  
  
    centers = kmeans.cluster_centers_  
  
    closest_indices = []  
    for i in range(self.k):  
        center = centers[i]  
        distances = np.linalg.norm(vectors - center, axis=1)  
        closest_index = np.argmin(distances)  
        closest_indices.append(closest_index)  
  
    return closest_indices
```

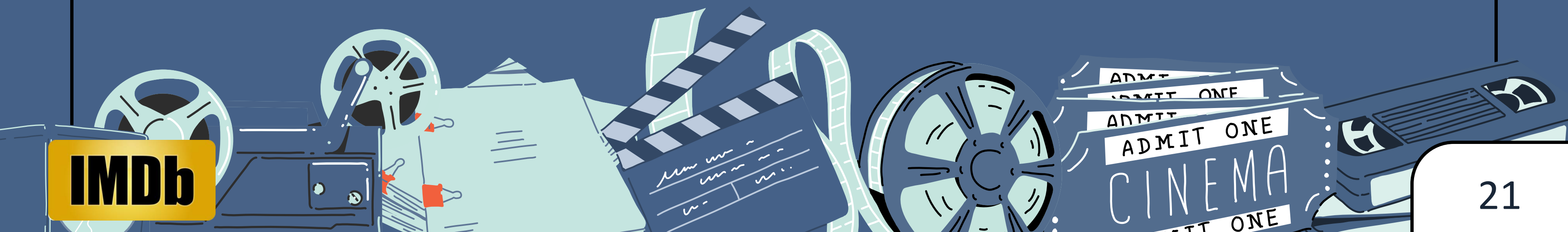
Review Summary

- Retrieve reviews that are closest to the centroid of each cluster.

```
def get_kmeans_reviews(self, movie_title, review_score):  
    db_path = os.path.join(self.db_dir, f"{movie_title}_{review_score}")  
    db = FAISS.load_local(  
        folder_path=db_path,  
        embeddings=self.embedding_model,  
        allow_dangerous_deserialization=True  
    )  
    docs = list(db.docstore._dict.values())  
    vectors = db.index.reconstruct_n(0, db.index.ntotal)  
    closest_indices = self.kmeans_clustering(vectors)  
    reviews = [docs[i].page_content.replace("\n", " ") for i in closest_indices]  
    return reviews
```

▼ faiss_db

- > American-Sniper_1
- > American-Sniper_2
- > American-Sniper_3
- > American-Sniper_4
- > American-Sniper_5
- > American-Sniper_6
- > American-Sniper_7
- > American-Sniper_8
- > American-Sniper_9
- > American-Sniper_10



Review Summary

- Summarizes movie reviews in Traditional Chinese.
 - Use Gemini API for text generation.
 - Custom prompt to ensure objectivity and clarity.
 - Highlights sentiment and main discussion points.

```
self.system_prompt = """
你是個專業的電影評論分析師，專長是閱讀多則關於同一部電影的評論，並從中提煉出最核心的觀點與整體評價。你的任務是接收一系列電影評論文字，並生成一段精簡、客觀、全面的摘要。

你的摘要必須：
1.  **使用繁體中文** 撰寫。這是最重要的要求。
2.  **反映整體情緒**：準確傳達評論者對電影的普遍看法（例如：壓倒性好評、褒貶不一、多數負評）。
3.  **點出關鍵優缺點**：總結評論中反覆提及的主要優點（如精彩的演技、驚人的視覺效果、深刻的劇情）和缺點（如拖沓的節奏、薄弱的角色、不合邏輯的情節）。
4.  **提及討論焦點**：如果評論集中討論某些特定面向（例如：導演風格、配樂、主題深度），應在摘要中提及。
5.  **保持中立客觀**：僅根據提供的評論內容進行總結，不加入個人意見或外部資訊。
6.  **語言精練易懂**：摘要應簡潔明瞭，避免冗長或使用過於專業的術語（除非評論普遍使用）。

請專注於綜合分析所有提供的評論，輸出一份高品質的繁體中文摘要。
"""

self.prompt_template = """
請根據以下多則關於電影「{movie_title}」的評論，生成一段符合要求的繁體中文摘要。

**評論列表：**

{reviews_list}

**繁體中文摘要：**
"""
```

```
response = self.client.chat.completions.create(
    model=self.model,
    messages=[
        {"role": "system", "content": self.system_prompt},
        {"role": "user", "content": final_prompt},
    ]
)
```


Review Summary

○ Summarization Result

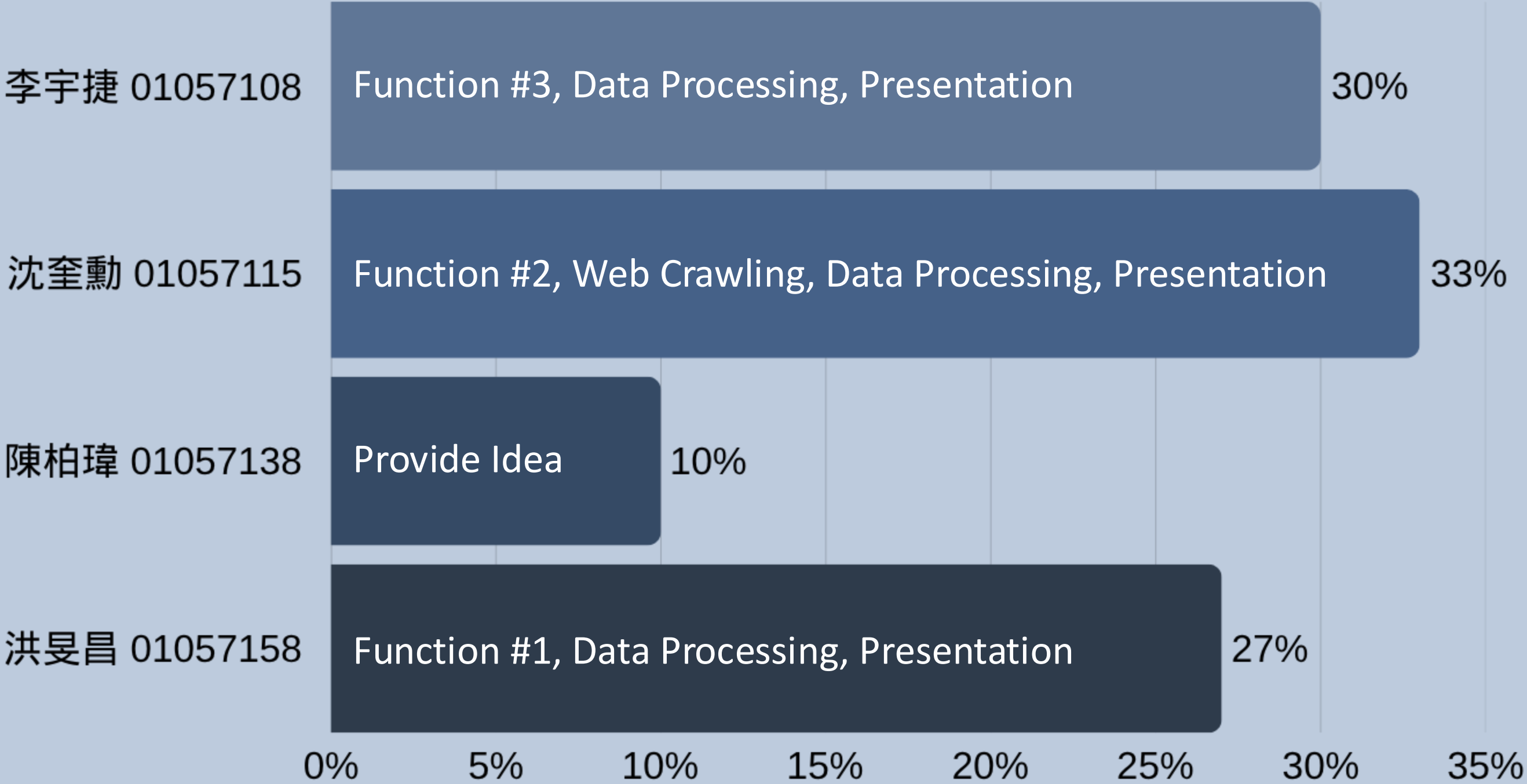
Movie: Oppenheimer

Source: 4,306 Reviews

Number of Clusters: 5

- "1": "根據這些評論，「Oppenheimer」普遍受到負評。許多評論者批評該片過於冗長、節奏緩慢，並將重點放在政治陰謀。
- "2": "多數評論對《奧本海默》的評價偏向負面，普遍認為該片過於冗長、沉悶，並缺乏焦點。許多評論者批評電影敘事結構複雜且難以理解。
- "3": "評論對於《奧本海默》的評價褒貶不一，但整體而言偏向負面。許多評論者認為電影過於冗長（近三小時），節奏拖沓，且缺乏情感深度。
- "4": "影評對《奧本海默》的評價褒貶不一，但普遍認為電影過於冗長，節奏拖沓是主要問題。許多評論者認為三個小時的片長對於一部歷史傳記片來說過長了。
- "5": "影評對諾蘭的《奧本海默》褒貶不一，雖然普遍讚賞演員的精湛演技（尤其是席尼·墨菲和羅伯特·道尼·Jr.），但對電影的敘事結構和節奏仍有不少批評。
- "6": "「Oppenheimer」的評價褒貶不一，主要爭議點在於其過長的片長和複雜的敘事結構。多數評論讚賞演員的精湛演技，但對電影的節奏和敘事結構仍有不少批評。
- "7": "影評對《奧本海默》的評價褒貶不一。普遍讚揚其演員表現，尤其是席尼·墨菲對奧本海默的詮釋，以及勞勃·道尼·Jr. 對阿諾·莫伊塞維奇的詮釋。
- "8": "影評普遍對《奧本海默》給予正面評價，盛讚席尼·墨菲（Cillian Murphy）對主角奧本海默的出色演繹，認為其演技精湛且富有層次感。
- "9": "影評普遍盛讚《奧本海默》是一部傑作，克里斯多夫·諾蘭的執導功力與席尼·墨菲的精湛演技是本片最受讚賞的亮點。
- "10": "影評普遍盛讚克里斯多夫·諾蘭的《奧本海默》是一部傑作，許多評論家認為它不僅是諾蘭的最佳作品，甚至是近年來最偉大的電影之一。

Work Contribution





Thank you

<https://github.com/shenks0628/Information-Retrieval-IMDb-project>

IMDb

END